

CHAPTER 1: INTRODUCTION

In today's fast-paced healthcare environment, an efficient appointment scheduling system is essential for both medical practitioners and patients. Our project aims to develop a user-friendly doctor appointment system that streamlines the booking process, reduces wait times, and enhances patient satisfaction. By integrating advanced features such as real-time availability, automated reminders, and user-friendly interfaces, the system will cater to the diverse needs of users, including doctors, nurses, and administrative staff. Additionally, it will facilitate better resource management for healthcare facilities, ultimately improving overall service delivery. With an emphasis on security and data privacy, our solution will ensure that patient information is protected while providing a seamless experience.

1.1 PURPOSE:

The purpose of the doctor appointment system project is to streamline the scheduling process for patients and healthcare providers, reducing wait times and improving efficiency. By offering real-time availability and automated reminders, it enhances patient satisfaction and ensures better resource management. Additionally, the system prioritizes data security and privacy, safeguarding sensitive patient information. Ultimately, it aims to improve overall healthcare delivery and accessibility.

1.2 SCOPE:

1.2.1 Current scope:

The current scope of the doctor appointment system project includes developing a web-based platform for easy scheduling, real-time availability, and automated reminders. It will support various healthcare providers in managing their appointments and patient data securely. Future enhancements may include telehealth integration and mobile app development for increased accessibility.

1.2.2 Future scope:

The future scope of the doctor appointment system includes integrating telehealth capabilities for virtual consultations and developing a mobile app for convenient scheduling. Additionally, implementing AI-driven analytics and integrating with electronic health records (EHR) will enhance resource management and improve patient outcomes.

1.3 DESCRIPTION:

The doctor appointment system is designed to streamline the scheduling process for patients and healthcare providers, enhancing overall efficiency in healthcare delivery. This web-based platform allows

patients to easily book, reschedule, or cancel appointments through a user-friendly interface. Real-time availability ensures that users can select convenient time slots, reducing wait times and minimizing scheduling conflicts.

Automated notifications and reminders help keep both patients and providers informed, improving attendance rates and reducing no-shows. The system securely stores patient information, ensuring compliance with data privacy regulations. Healthcare providers can manage their appointments efficiently, monitor patient flow, and optimize their schedules.

The platform supports multiple healthcare providers, including clinics and individual practitioners, allowing for flexibility in managing various appointment types. Future enhancements may include telehealth capabilities, enabling virtual consultations and expanding access to care. A mobile application will provide users with on-the-go scheduling and reminders, increasing convenience.

Additionally, integrating advanced analytics will offer insights into patient trends and resource utilization, helping providers make informed decisions. By focusing on user experience and security, the doctor appointment system aims to improve patient satisfaction and foster better communication between patients and healthcare professionals.

1.4 OBJECTIVES:

The primary objectives for a travel and tourism website typically include:

- 1. Information dissemination:** Information dissemination for the doctor appointment system will involve user guides, tutorials, and FAQs to assist patients and providers. Additionally, workshops, social media campaigns, and email newsletters will promote features and gather feedback for continuous improvement..
- 2. Booking facilities:** The booking facilities of the doctor appointment system will enable patients to view real-time availability, book, reschedule, or cancel appointments online. Automated confirmations and reminders will enhance convenience, while options for both in-person and telehealth visits will cater to diverse patient needs.
- 3. User engagement:** User engagement for the doctor appointment system will be enhanced through personalized reminders and a user-friendly interface that encourages feedback. Regular updates and educational content will keep users informed and motivated, fostering a stronger connection between patients and healthcare providers.
- 4. Conversion:** Conversion for the doctor appointment system involves turning interested users into active bookers through an optimized user experience and clear calls to action. Offering incentives like first-time booking discounts can further encourage users to schedule appointments, boosting overall

engagement with the platform.

5, Community building: Community building for the doctor appointment system focuses on creating a supportive network among patients and healthcare providers. This can be achieved through online forums, social media groups, and regular webinars where users can share experiences, ask questions, and access valuable health information.

CHAPTER 2 : TOOLS AND TECHNOLOGIES

○ TECHNOLOGY

2.1.1 Front-end technologies:

CSS: Cascading Style Sheets, or CSS, is what gives our HTML visual appeal and draws in the user. To put it simply, style sheets dictate the presentation of HTML elements on a page.

CSS is what makes everything not look like a white background with a bunch of Times New Roman texts and blue hyperlinks. Have you ever tried to load Amazon on a bad day and see nothing but a white page with a huge list of black texts, some blue links and nothing else? That's Amazon loading without styles. Styles are what transition beautiful designs onto a site.

A new trend with styling is to use what is called a CSS pre-processor. These include Less, Sass and stylists. Pre-processors are scripting languages that compile to CSS for the browser and are very popular as they expedite the development process. They accommodate some programming logic.

CSS pre-processors consolidate styling through “usability” and scale easily for use with large websites. Most front-end engineers of all levels have experience with Pre-processors due to their power and flexibility to suit all scenarios, from building small single-page websites to enterprise applications such as Groupon or NBCUniversal.

JavaScript: JavaScript is a runtime language for web browsers. This means that when you open a web page, the page will load both the foundational JavaScript that is standard with the page and any new JavaScript added to a page. The new JavaScript will load in parallel with it and can perform actions and make decisions.

JavaScript differentiates from back-end languages such as Java or Ruby in that it's visible to the user and is not compiled ahead of time to run secretly behind the curtain. It's the true programming language of front-end engineering and the underlying language that ties everything together.

JavaScript is a powerful language primarily used for creating interactive web applications, allowing developers to manipulate HTML and CSS dynamically on the client side. It also powers server-side development through Node.js, enabling the creation of APIs and managing databases. Additionally, JavaScript is utilized in mobile app development with frameworks like React Native and even in game development with libraries like Phaser.

React js :React.js is a popular JavaScript library used for building user interfaces, particularly for single-

page applications. Developed by Facebook, it allows developers to create reusable UI components, making the code more maintainable and efficient. React uses a virtual DOM to optimize rendering, improving performance by only updating parts of the UI that have changed. Its component-based architecture promotes a clear structure and facilitates easier debugging.

React.js is an open-source JavaScript library that revolutionizes the way developers build user interfaces by emphasizing a component-based architecture. Each component manages its own state and can be reused throughout the application, leading to a more organized and modular codebase.

■ **BACK-END TECHNOLOGY:**

● **Node js :** Node.js is a powerful, open-source runtime environment that allows developers to execute JavaScript code on the server side. Built on Chrome's V8 JavaScript engine, Node.js is designed for building scalable and high-performance applications, particularly for real-time web services like chat applications and APIs. Its non-blocking, event-driven architecture enables it to handle multiple connections simultaneously, making it ideal for applications that require frequent data updates. With a vast ecosystem of libraries and frameworks available through npm (Node Package Manager), developers can easily integrate various functionalities into their applications, streamlining the development process. Node.js is particularly favored for its ability to use the same language JavaScript across both the client and server, enhancing code consistency and developer productivity.

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine, designed for developing server-side and networking applications. Its non-blocking, event-driven architecture allows it to handle many simultaneous connections efficiently, making it particularly well-suited for I/O-intensive tasks like real-time applications, web servers, and APIs. The single-threaded model utilizes asynchronous programming, which enables high throughput and scalability without the overhead of traditional multi-threading.

With npm (Node Package Manager), Node.js boasts a rich ecosystem of libraries and frameworks, such as Express.js for building web applications, Socket.io for real-time communication, and Sequelize for database management. This versatility allows developers to create a wide range of applications—from microservices to full-stack solutions—while leveraging the same language for both client-side and server-side code.

- **Express.js :** Express.js is a minimalist web framework for Node.js that simplifies the process of building web applications and APIs. It provides a robust set of features, including middleware support, routing, and template rendering, which allows developers to create server-side applications quickly and efficiently. With its flexible architecture, Express can easily integrate with various databases and third-party middleware, enabling functionality such as authentication,

logging, and error handling. The framework is known for its performance and scalability, making it a popular choice for building RESTful APIs and single-page applications. Its straightforward syntax and extensive documentation make it accessible for both beginners and experienced developers.

Express.js is a powerful and flexible web application framework for Node.js, designed to streamline the process of building server-side applications. It offers a minimalistic approach while providing essential features such as robust routing, middleware support, and integration with various templating engines, making it easy to manage request and response cycles. Express's middleware functions allow developers to define custom logic that can be executed during the request-handling process, enabling functionalities like authentication, data parsing, and session management. With built-in support for RESTful APIs, Express makes it simple to set up routes that correspond to different HTTP methods, ensuring clean and organized code.

Its vast ecosystem, including tools like Mongoose for MongoDB interaction and Passport.js for authentication, enhances its capabilities, allowing for the creation of complex applications with relative ease. The active community and wealth of plugins contribute to its popularity in the Node.js landscape, making it an excellent choice for both small projects and large-scale applications.

○ **PROJECT PLANNING:**

- **Project development approach :**
 - **Planning:** Define project goals, scope, and requirements. Identify stakeholders and gather feedback to establish a clear vision.
 - **Design:** Create wireframes and architecture for the application, focusing on user experience (UX) and technical design.
 - **Development:** Break the project into manageable tasks, using an agile or waterfall methodology. Code the application using the chosen technologies, such as JavaScript, React, or Node.js.
 - **Testing:** Conduct unit, integration, and user acceptance testing to identify and fix bugs, ensuring the application meets requirements.
 - **Deployment:** Launch the application to a production environment, making it accessible to users.
 - **Maintenance:** Monitor performance, gather user feedback, and implement updates or new features as needed.

2.1 MILESTON AND DELIVERABLES:

Month1:- Milestones & Deliverables

Milestones	Deliverables
Study about our web application requirements, planning.	Analysis Report
Understand a project definition, basic term and logic for parameter evaluation.	Analysis Report
Gathering the requirements of the project using different fact finding technique.	Analysis Report
Still Continue with Requirement's study.	Analysis Report

Month2:- Milestones & Deliverables

Milestones	Deliverable
System analysis	Analysis Report
System Design including various diagrams	SRS

Month3:-Milestones & Deliverables

Milestones	Deliverables
Integrating techniques of php and database	Designing/Coding
Database creation and Procedures	Designing/Coding
Admin Module of Client Support System	Designing/Coding
Consultant Module of Client Support System	Designing/Coding
Accountant Module of Client Support System	Designing/Coding

Month4:-Milestones & Deliverables

Milestones	Deliverables
Client Module of Client Support System	Designing/Coding
Website Testing	Testing
Required changes after testing	Designing/Coding

2.1.1 : Group Dependencies:

The team structure depends on the management style of the organization, number of people in the team, their skill levels and the problem difficulty. Our team organization is *democratic decentralized* which doesn't have a team leader. Decision is made by all of us and the problems were discussed and solved by all of us after consulting and discussing with our external guide and project guides.

2.2 Project Scheduling

Project scheduling involves separating the total work in a project into separate activities and judging the time required to complete these activities. Usually, some of these activities are carried out in parallel.

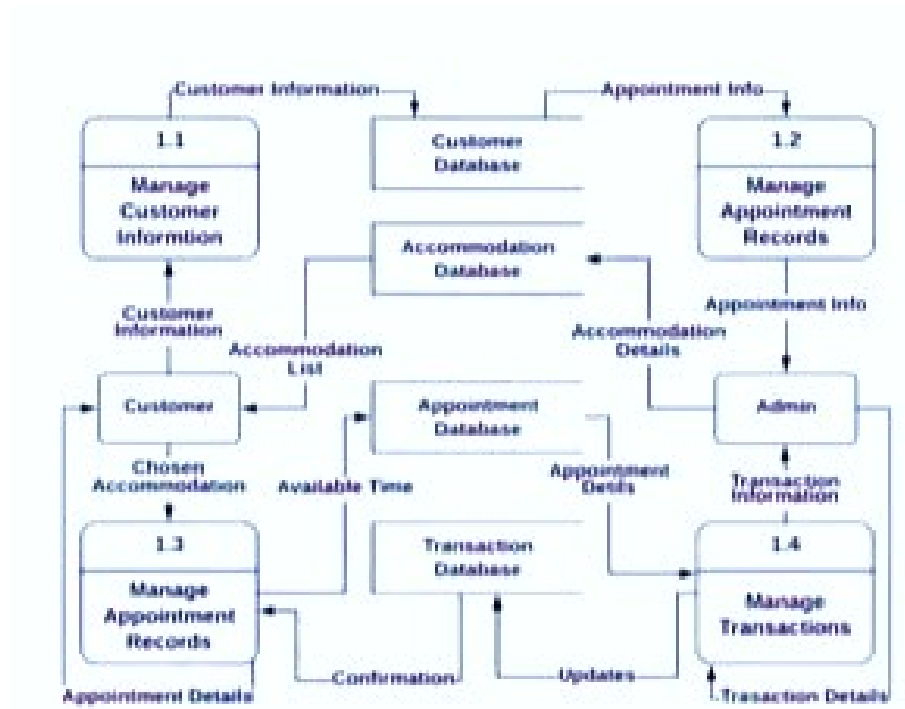
- Work Breakdown Structure

Work Breakdown Structure is used to decompose a given task set recursively into small activity [Fig (2.3)].

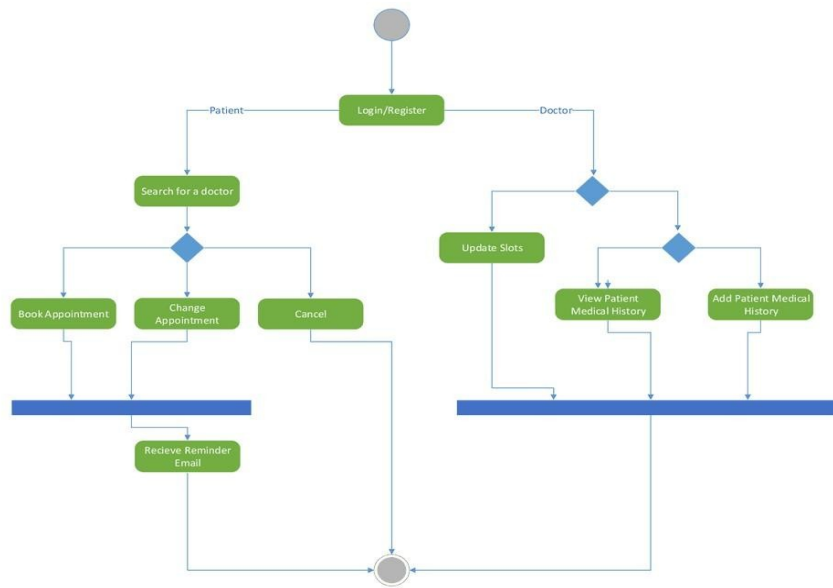
- Activity Network Representation

An activity network shows the different activities making of a project, their estimated duration and interdependencies [Fig (2.4)].

a) Work breakdown structure: Figure-(2.4.1)



b) Activity Network Representation : Figure-(2.4.2)



CHAPTER 3 SYSTEM REQUIREMENTS STUDY

3.1 USER CHARACTERISTICS:

- **Demographics:** Age, gender, education, income level, and occupation, which help in understanding the audience's background.
- **Psychographics:** Interests, values, lifestyle choices, and attitudes that influence how users engage with products.
- **Technical Proficiency:** Familiarity with technology and digital tools, which affects how users interact with applications.
- **User Needs and Goals:** Specific needs, motivations, and goals that drive user behavior and influence design decisions.
- **Behavior Patterns:** Typical usage patterns, such as frequency of use, preferred devices, and interaction methods.
- **Accessibility Requirements:** Considerations for users with disabilities, ensuring the product is usable for all.
- User must have basic knowledge of Computers.
- User should understand the use of all modules.
- User can easily interact with the proposed system.
- User must know the technical terms used in the company for performing different tasks specially related to call logs, payment details, transportation details and report retrieval.
- User should be also being aware about the running process of the system.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS:

- Software and Hardware Requirements are used to describe the minimum hardware and software requirements to run the Software. These requirements are described below.

3.2.1 Software Requirements:

Client:

- Web Browser: Any Compliant Browser
- Operating System: or Linux Windows

Server:

- Operating System: WINDOWS or LINUX SERVER OS

3.2.2 Hardware Requirements Client:

- 1.6 GHZ CPU
- 256 MB of RAM
- INTERNET CONNECTION
- Monitor
- Keyboard/Mouse
- Printer

3.1.1 Server:

- 1.6 GHZ CPU
- 1 GB RAM
- INTERNET CONNECTION
- Monitor

- Keyboard/Mouse
- Printer

3.2.1 Functional Requirements:

- User Interfaces: Specifications for how users will interact with the system, including layouts, buttons, forms, and navigation paths.
- Authentication and Authorization: Requirements outlining how users will log in, what credentials are needed, and what permissions different user roles will have.
- Data Management: Specifications for data creation, retrieval, updating, and deletion (CRUD operations). This includes how data will be stored, managed, and manipulated within the system.
- Business Logic: Rules and calculations that dictate how the system processes data, including workflows and decision-making processes.
- System Integration: Requirements for how the software will interact with other systems, such as APIs, databases, and third-party services.
- Reporting and Analytics: Specifications for generating reports, visualizations, and insights from the data, detailing what metrics and KPIs need to be tracked.
- Notifications and Alerts: Requirements for how the system will communicate with users, including email notifications, alerts, and messages based on specific triggers.
- Error Handling: Specifications for how the system should respond to errors, including user notifications and logging.
- Performance Requirements: Functional requirements that specify acceptable performance levels for various system operations, such as response time and processing speed.

3.3.1 None Functional Requirements:

- Performance: Defines how the system should perform under specific conditions, including response time, throughput, and resource usage. For example, a requirement might state that a webpage should load within two seconds.
- Scalability: Specifies the system's ability to handle increased loads, such as more users or transactions, without compromising performance.

- **Security:** Outlines the measures needed to protect the system from unauthorized access, breaches, and data loss. This can include encryption standards, user authentication methods, and compliance with regulations.
- **Usability:** Focuses on the user experience, detailing how intuitive and easy to use the software should be. This can include requirements for user interface design, accessibility standards, and user satisfaction metrics.
- **Reliability:** Describes the system's ability to operate consistently over time, including availability and fault tolerance. A requirement might specify that the system should have 99.9% uptime.
- **Maintainability:** Addresses how easy it is to update, fix, or improve the software. This can include requirements for modularity, documentation, and adherence to coding standards.
- **Portability:** Specifies how easily the software can be transferred from one environment to another, such as different operating systems or hardware platforms.
- **Compliance:** Ensures that the software adheres to relevant laws, regulations, and industry standards, such as GDPR for data protection or HIPAA for healthcare.

3.3 CONSTRAINTS:

Technical Constraints: Limitations related to technology choices, such as specific programming languages, frameworks, or platforms that must be used. For example, a system may need to be built using a particular database or run on specific server configurations.

Performance Constraints: Requirements that define acceptable performance limits, such as maximum load times, response times, and throughput. For instance, a web application may need to support 1,000 concurrent users without degradation in performance.

Regulatory and Compliance Constraints: Legal requirements that the software must adhere to, such as data protection laws (e.g., GDPR) or industry standards (e.g., HIPAA for healthcare applications).

Budget Constraints: Financial limitations that affect resource allocation, including development costs, licensing fees, and operational expenses. This may restrict the choice of technologies or the scope of the project.

Time Constraints: Deadlines that must be met for project delivery, which can influence the development process, resource allocation, and prioritization of features.

User Constraints: Limitations based on user characteristics, such as accessibility requirements for users with disabilities or specific hardware limitations for the target audience.

Environmental Constraints: Conditions related to the operational environment of the system, such as network bandwidth, hardware specifications, or integration with existing systems.

3.4 ASSUMPTION AND DEPENDENCIES:

3.4.1 Assumption:

1. User Availability: Assuming users will be available for testing and feedback during specified phases of development.
2. Resource Availability: Assuming that necessary resources, such as development tools or skilled personnel, will be available throughout the project.
3. Technology Stability: Assuming that the chosen technologies (e.g., frameworks, libraries) will remain stable and supported during the project lifecycle.
4. Market Conditions: Assuming that market conditions or user needs will remain relatively constant during the development period.

3.4.2 Dependencies:

1. Technical Dependencies: Certain features may depend on the completion of other features or components. For example, a user authentication system must be in place before user profile features can be developed.
2. Third-Party Dependencies: Relying on external services, APIs, or libraries can introduce dependencies that affect the project timeline and functionality.
3. Team Dependencies: Team members may have overlapping responsibilities, meaning the completion of one task may depend on another team member's work.
4. Regulatory Dependencies: Compliance with specific regulations may require certain functionalities to be developed before other

CHAPTER 4 SYSTEM ANALYSIS

4.1 STUDY OF CUURENT SYSTEM:

System Overview: A brief description of the existing system, including its purpose, key features, and user base. This may include the technologies used and the architecture of the system.

User Feedback: Gathering input from users through surveys, interviews, or usability testing to understand their experiences, pain points, and satisfaction levels. This helps identify areas that require enhancement or redesign.

Functional Analysis: Reviewing the current system's functionalities to determine how well it meets user needs and requirements. This includes assessing how efficiently tasks are completed and whether any critical functionalities are missing.

Performance Evaluation: Analyzing system performance metrics such as response time, uptime, and resource utilization. This helps identify bottlenecks or issues that may affect user experience.

Security Assessment: Evaluating the current system's security measures to identify vulnerabilities and ensure compliance with relevant regulations and standards.

Technical Debt: Identifying outdated technologies, poorly written code, or architectural flaws that could hinder future development and maintenance.

Integration Points: Understanding how the current system integrates with other systems or services, and identifying any challenges or limitations in these integrations.

Documentation Review: Examining existing documentation to determine its completeness and accuracy, which is vital for maintenance and onboarding new team members.

4.2 MODULS ANF FUNCTIONALITY OF PROPOSED ACTIVITY:

User Management Module:

- **Functionality:** Handles user registration, login, and authentication. Includes features for user profile management, role assignments, and password recovery.

Dashboard Module:

- **Functionality:** Provides an overview of key metrics and data visualizations. Users can access personalized dashboards to view relevant information at a glance.

Data Management Module:

- **Functionality:** Facilitates data entry, storage, retrieval, and manipulation. Supports CRUD (Create, Read, Update, Delete) operations for various data types, including validation and error handling.

Reporting Module

- **Functionality:** Generates reports based on user-defined criteria. Allows users to create, export, and schedule reports in various formats (e.g., PDF, Excel).

Integration Module

- **Functionality:** Manages connections to external systems and APIs. Ensures smooth data exchange between the proposed system and other platforms or services.

Notification Module

- **Functionality:** Sends alerts and notifications to users based on specific events or actions. Supports email, SMS, or in-app notifications.

Settings and Configuration Module

- **Functionality:** Allows administrators to configure system settings, manage application preferences, and control feature access based on user roles.

Audit and Logging Module

- **Functionality:** Tracks user actions and system events for security and compliance purposes. Provides an audit trail that can be reviewed by administrators.

Help and Support Module

- **Functionality:** Offers resources for user assistance, including FAQs, documentation, and support ticket submission. Facilitates user feedback collection.

4.3 Feasibility study:

Technical Feasibility:

- **Assessment of Technology:** Evaluates whether the current technology infrastructure can support the proposed system. This includes analyzing hardware, software, and network requirements.
- **Expertise and Resources:** Determines if the organization has the necessary technical skills and resources to develop and maintain the system.

Economic Feasibility:

- **Cost-Benefit Analysis:** Compares the projected costs of the project against the expected benefits, including potential revenue, cost savings, and productivity improvements.
- **Budgeting:** Estimates overall project costs, including development, implementation, and ongoing maintenance expenses.

Operational Feasibility:

- **User Acceptance:** Assesses the likelihood of user adoption and satisfaction with the proposed system. This may involve gathering user feedback and evaluating their willingness to transition to the new system.
- **Impact on Current Operations:** Analyzes how the new system will affect existing workflows, processes, and organizational structure.

Legal and Regulatory Feasibility:

- **Compliance Requirements:** Identifies any legal or regulatory obligations that must be met, such as data protection laws or industry-specific regulations.
- **Intellectual Property Issues:** Evaluates potential intellectual property considerations, such as software licensing and patent implications.

Schedule Feasibility:

- **Timeline Estimation:** Evaluates whether the project can be completed within the desired timeframe. This includes assessing key milestones and deadlines.
- **Resource Availability:** Considers whether the necessary resources (human, technical, financial) will be available when needed.

Risk Assessment:

- **Identification of Risks:** Identifies potential risks associated with the project, including technical challenges, budget overruns, and user resistance.
- **Mitigation Strategies:** Proposes strategies to minimize identified risks, ensuring that contingency plans are in place.

4.4 Requirements validation:

Review and Inspection:

- Conduct formal reviews of the requirements documentation with stakeholders, including users, developers, and project managers. This helps identify any ambiguities, inconsistencies, or missing information.

Stakeholder Feedback:

- Gather feedback from stakeholders through interviews, surveys, or workshops. Engaging users directly helps validate that their needs are accurately captured and prioritized.

Prototyping:

- Develop prototypes or mockups of the system to visualize how requirements will translate into functionality. This allows stakeholders to interact with a preliminary version of the product and provide feedback.

Use Cases and Scenarios:

- Create use cases or user stories that illustrate how users will interact with the system. This helps ensure that functional requirements align with real-world scenarios and user workflows.

Traceability:

- Establish traceability matrices that link requirements to their sources (e.g., user needs, business objectives) and to design elements. This ensures that all requirements are accounted for and can be tracked throughout the development process.

Testability:

- Review requirements to ensure they are measurable and testable. This means defining acceptance criteria for each requirement, so it's clear how success will be determined.

Conflict Resolution:

- Identify and resolve any conflicting requirements or priorities among stakeholders. Facilitating discussions can help reach a consensus on the most critical needs.

Change Management:

- Implement a process for managing changes to requirements. This ensures that any modifications are properly documented, reviewed, and communicated to all stakeholders.

4.5 CLASS DIAGRAM:

- A class diagram is a graph of classified elements connected by their various static relationships. It is shown here for Client Support System. This includes the System and the End-users as its main classes. Here are three different fields: Class, Attributes, and Operations.
- Class shows the class name, i.e., System, End-user, Data and Sign Clip. They are connected with each other through links and their relation with each other is shown through the numbers represented on the link; here * indicates zero or more multiplicity. Here Data class is connected to System through a Composition link which is the collaboration of all participants are part of one composite class. Attributes provide the details of the Class while Operations show all possible operations respective class can do in the system.

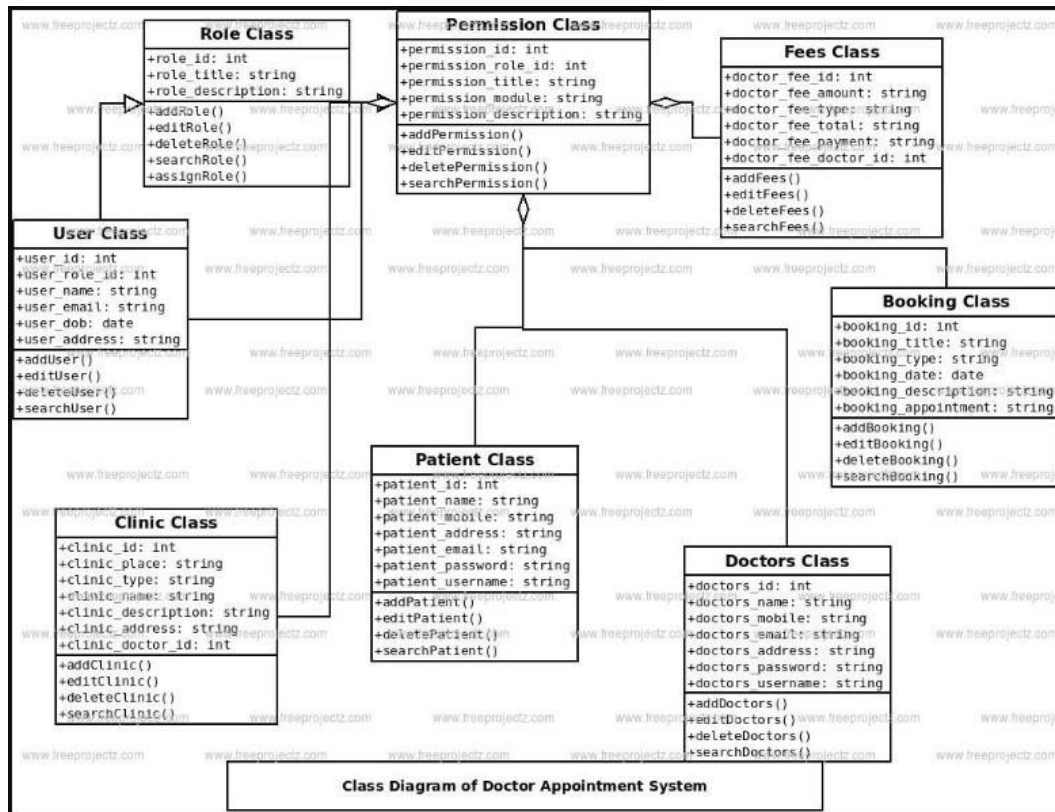


Figure – (4.5)

4.5 SYSTEM ACTIVITY:

- A use case diagram shows the relationship among actors and use cases within a system. Hence it provides the characteristics of the actors whose behavior and relationships can be well understood using the diagrams elaborated here.
- An end-user can perform various tasks on the application; he may use signs or upload a new sign on his own. On the other hand, the system would respond to the operations done by the user. It would display the sign clips and store the uploaded sign to the defined category. Also it would provide a message whenever needed for confirmation.
- Here the rectangle indicates system boundary, out of which there are actors found who perform

various operations on the system which are the end-user and the system here. An elliptical shape shows the use-case while the connecting links between an actor and a use case are said to be communicates.

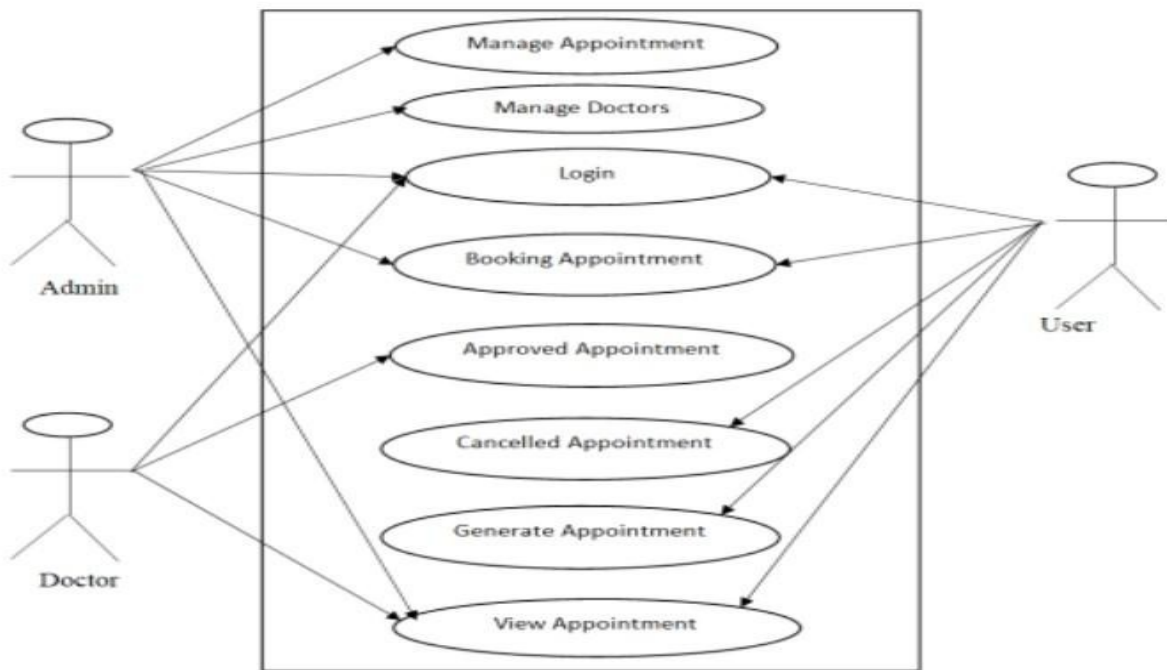


Figure-4.5

➤ Use Case Diagram's:

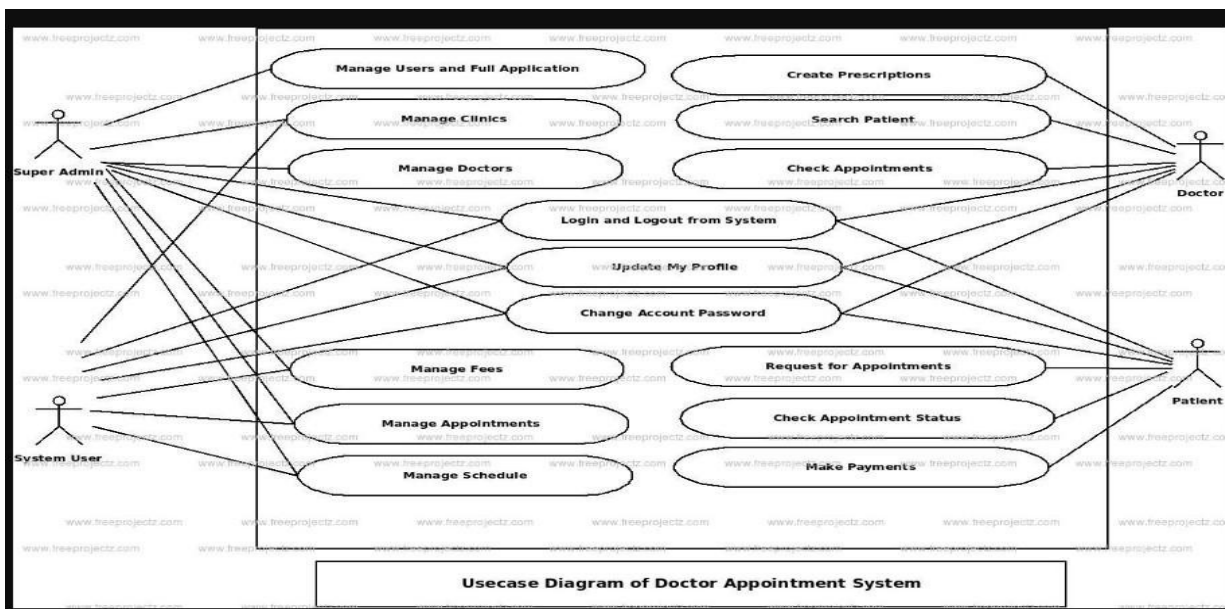


Figure-4.5

➤ Use case Diagram no:2

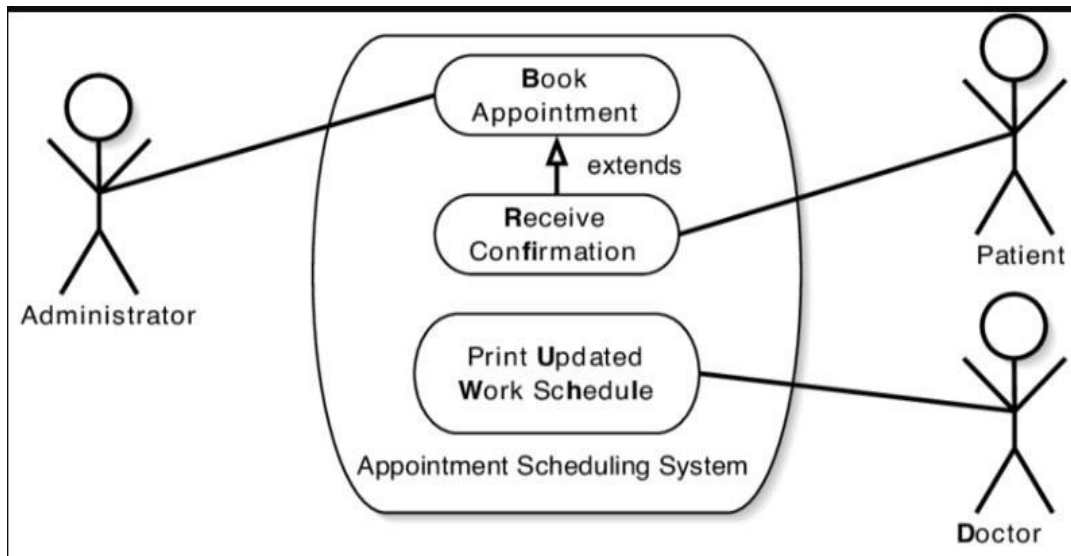


Figure-4.5.1

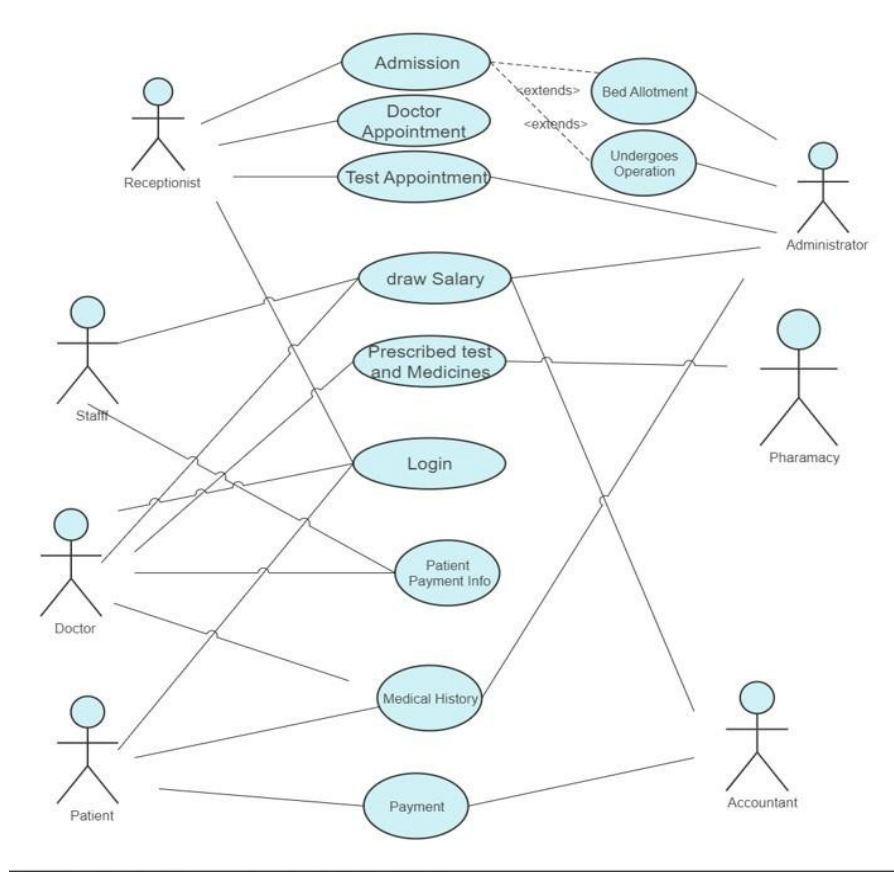
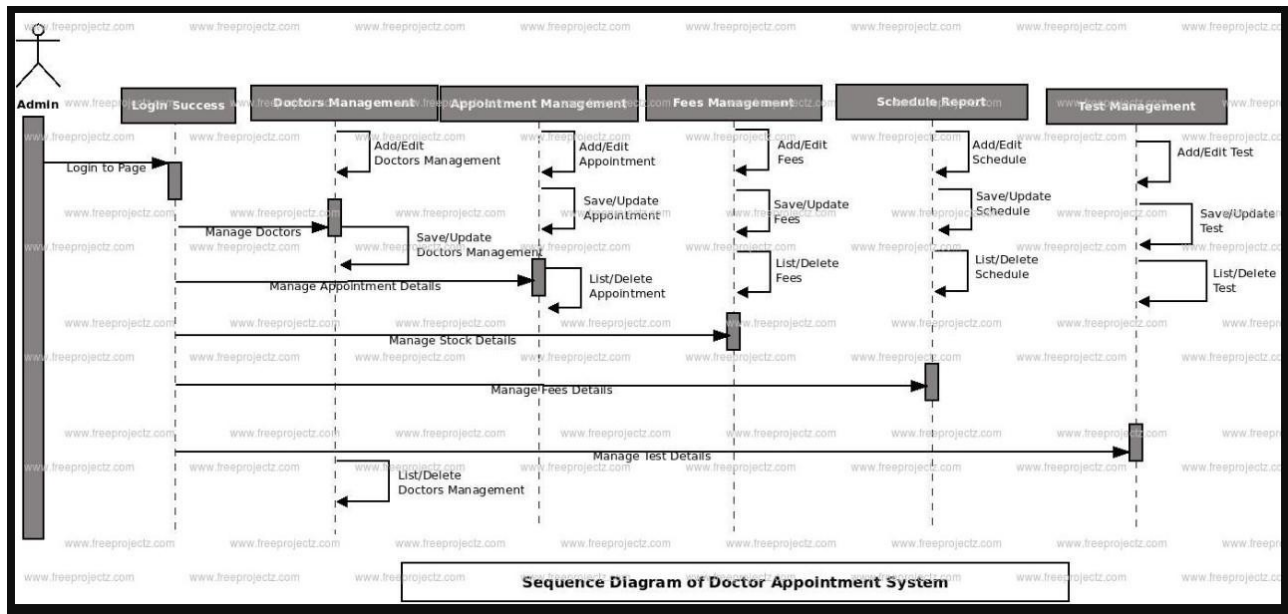


Figure-4.5.2

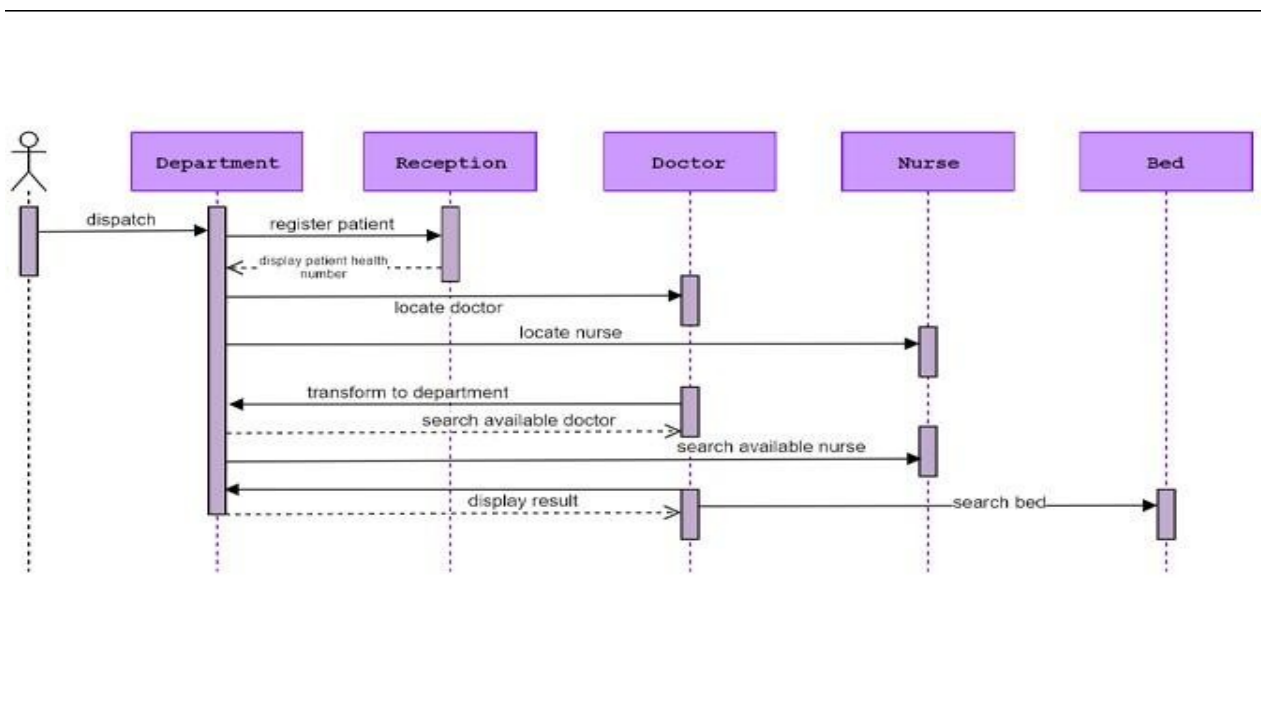
4.6 Sequence Diagram

➤ A sequence diagram represents an Interaction, which is a set of messages exchanged among objects within collaboration to effect a desired operation or result. Here are the sequence diagrams for various interactions among the user, system and the data storage. It must be noted that the rectangle box on the top of the diagram indicates the object or actor and dashed lines beneath to it shows an object's lifeline. Another rectangles following and followed by the dashed lines in a vertical manner show the activation period of the object or actor when it performs some actions. A solid arrow conveys a message while the dashed arrow gives return message. These message names are written along with their respective arrows as shown further in the diagram.

4.6.1 Sequence Diagram :



4.6.2 Sequence Diagram:



CHAPTER 5: SYSTEM DESIGN

5.1 DATA BASE DESIGN:

5.1.1 Login details:

Field Name	DATATYPE	DESCRIPTION
user_id	INT	Primary key, unique identifier
username	VARCHAR(50)	Unique username for each user
password_hash	VARCHAR(255)	Hashed password for security
sign_up_date	DATETIME	Date and time of account creation
user_type	ENUM('patient', 'doctor')	Role of the user

5.1.2 patient details:

Field Name	Data type	Description
patient_id	int	Primary key Uniqueidentifier
first_name	Varchar(50)	Patient's first name
last_name	Varchar(50)	Patient last name
date_of_birth	DATE	Patient Date of Birth
gender	Enum('Male','Female',' other')	Patient Gender
phone_number	Varchar(15)	Patient contact number
email	Varchar(100)	Patient Email address
address	Varchar(255)	Patient Residential address
signup_date	DateTime	Date and Time of account creation

5.1.3 Client contact details:

Field	Data Type	Length	Constraint	Reference Table
client_contact_id	varchar2	10	Primary Key	
client_id	varchar2	10	Foreign Key	Client Details
first_name	varchar2	30	not null	
last_name	varchar2	30	not null	
Designation	varchar2	50	not null	
Email	varchar2	50	not null	
contact_number	number	12	not null	

5.1.4 Call Log Details:

- The following table is used to store call log details:

Field	Data Type	Length	Constraint	Reference Table
call_log_id	Number	4	Primary Key	
log_date	varchar2	20	not null	
start_time	varchar2	20	not null	
end time	varchar2	20	not null	
client_id	varchar2	10	Foreign key	Client Details
support_requested	varchar2	30		
consultant_id	varchar2	10	Foreign key	
call_type	varchar2	20	not null	Consultant Details
problem_assessment	varchar2	1000	not null	
actions_taken	varchar	1000	not null	

	r2			
recommendations	varchar2	1000		
support_type	varchar2	10	not null	
assessment_status	varchar2	10	not null	
Failed Reason	varchar2	250		
Pending Reason	varchar2	250		

5.1.5 AppointmentDetails:

- The following table is used to store transportation details of a call log:

Field	Data Type	Length	Constraint	Reference Table
Appointment_id	Number	4	Primary Key	
Date	Date		not null	
call_log_id	varchar2	10	Foreign Key	Call Log D

				et ails
Type	varchar2	10	not null	

5.1.6 Consultant payment details:

- Table 5.6 Transportation Details The following table is uses to store consultant payment details:

Field	Data Type	Length	Constraint	Reference Table
consultant_payment_id	Number	4	Primary Key	
payment_date	Date		not null	
consultant_id	varchar2	10	Foreign Key	Consultant Details
client_id	varchar2	10	Foreign Key	Client Details
payment_type	varchar2	30	not null	
Amount	Number	6	not null	

5.1.7 Client payment details:

Table 5.7 Consultant Payment Details The following table is uses to store client payment details:

Field	Data Type	Length	Constraint	Reference Table
client_payment_id	Number	4	Primary Key	
payment_date	Date		not null	
client_id	varchar2	10	Foreign Key	Client Details
payment_type	varchar2	30	not null	
Amount	Number	6	not null	
service tax	Number	6	not null	

5.1.8 Registered issues:

- The following table is used to register the issues of client companies:

Field	Data Type	Length	Constraint	Reference Table
issue_register_id	Number	4	Primary Key	
client_id	varchar2	10	Foreign Key	Client Details
Problem_description	varchar2	200	not null	
preferred_consultant	varchar2	10	Foreign Key	Consultant Details
Issue_title	varchar2	75	not null	

5.1.9 Schedule Task:

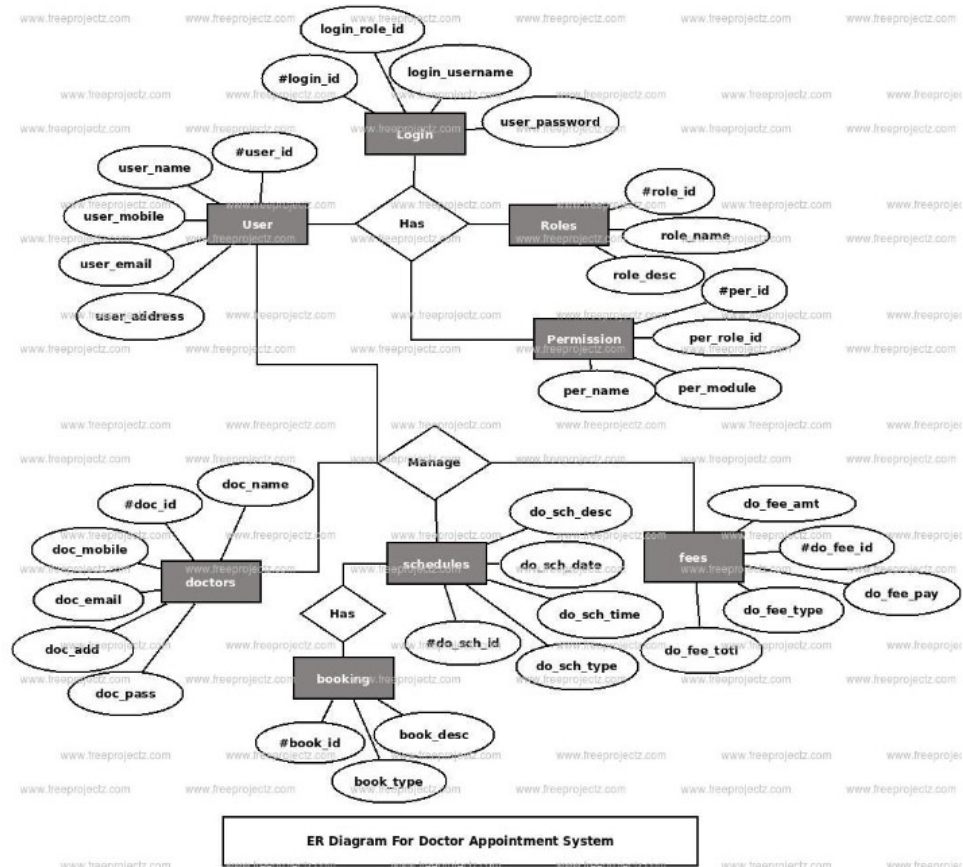
Table 5.9 Registered Issue:

- The following table is used to register the issues of client companies:

Field	Data Type	Length	Constraint	Reference Table
Sch_id	Number	4	Primary Key	
Consultant id	varchar2	10	Foreign Key	Consultant Details
Description	varchar2	200	not null	
Date	varchar2	10	Foreign Key	

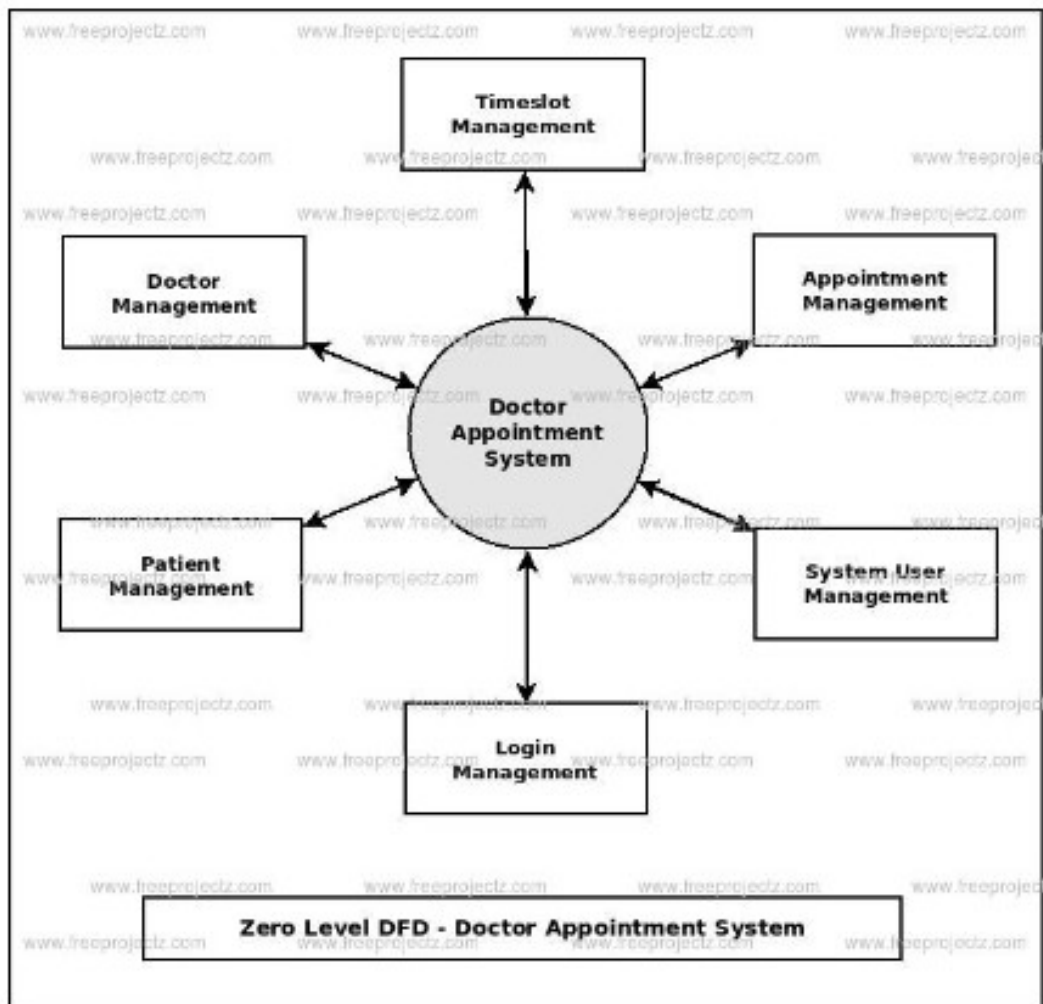
Title	varchar r2	75	not null	
-------	---------------	----	-------------	--

5.1 Entity Relationship Diagram:

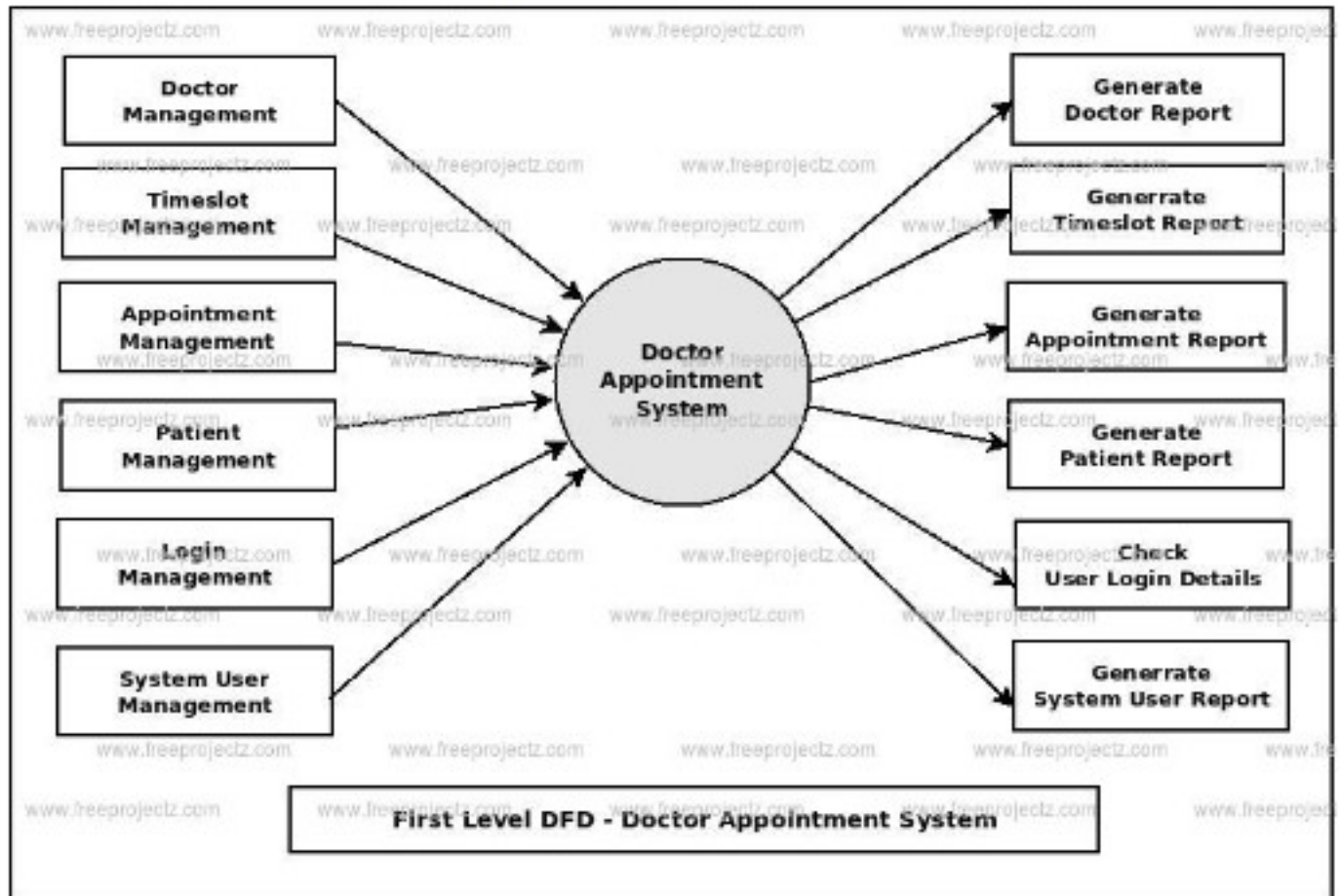


5.2 Data Flow Diagram:

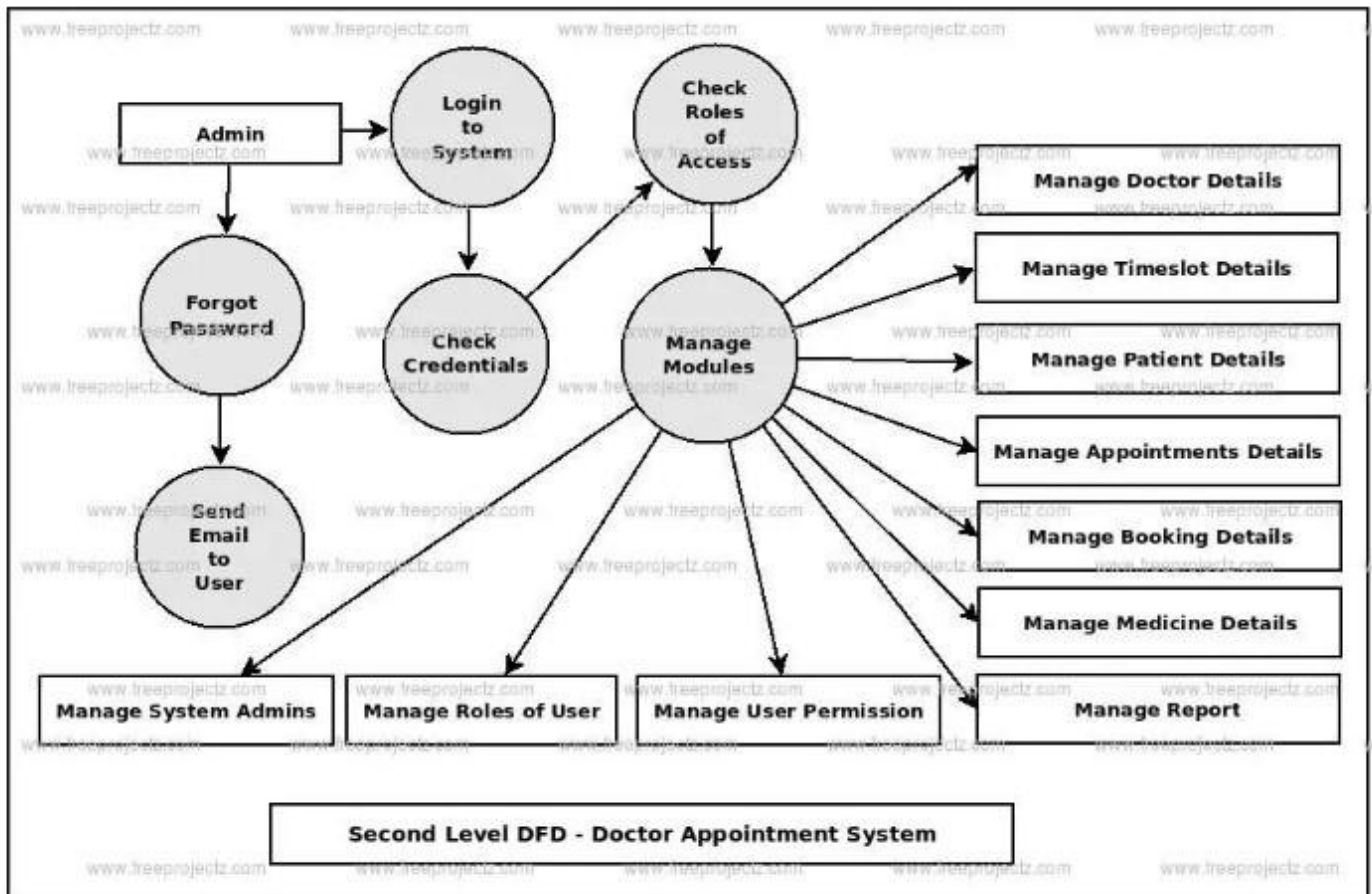
5.2.1 Level 0 DFD for Doctors Appointment System:



5.2.2 Level 1 DFD for Doctors Appointment System:



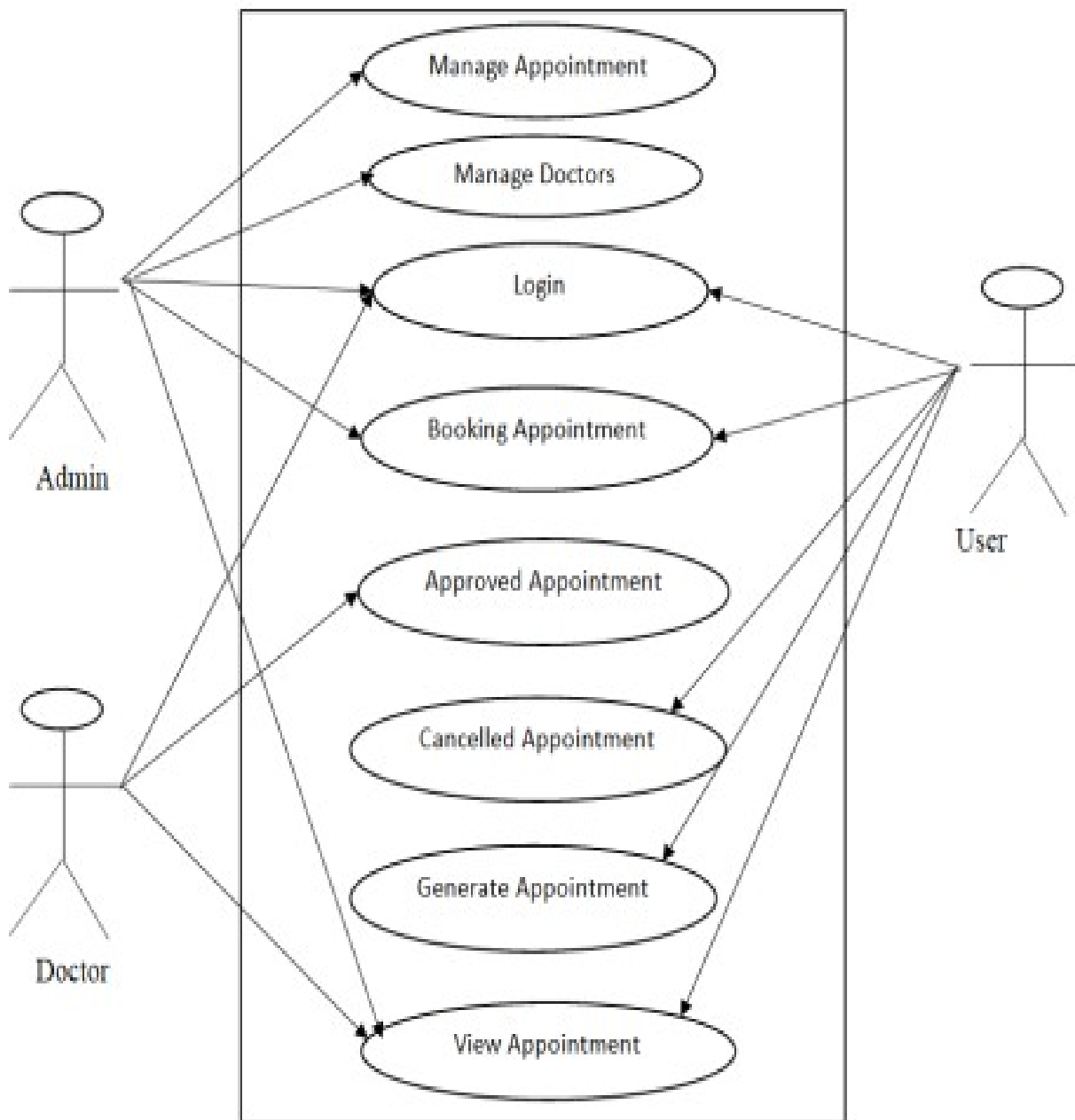
5.2.3 Level 2 DFD for Doctors Appointment System:



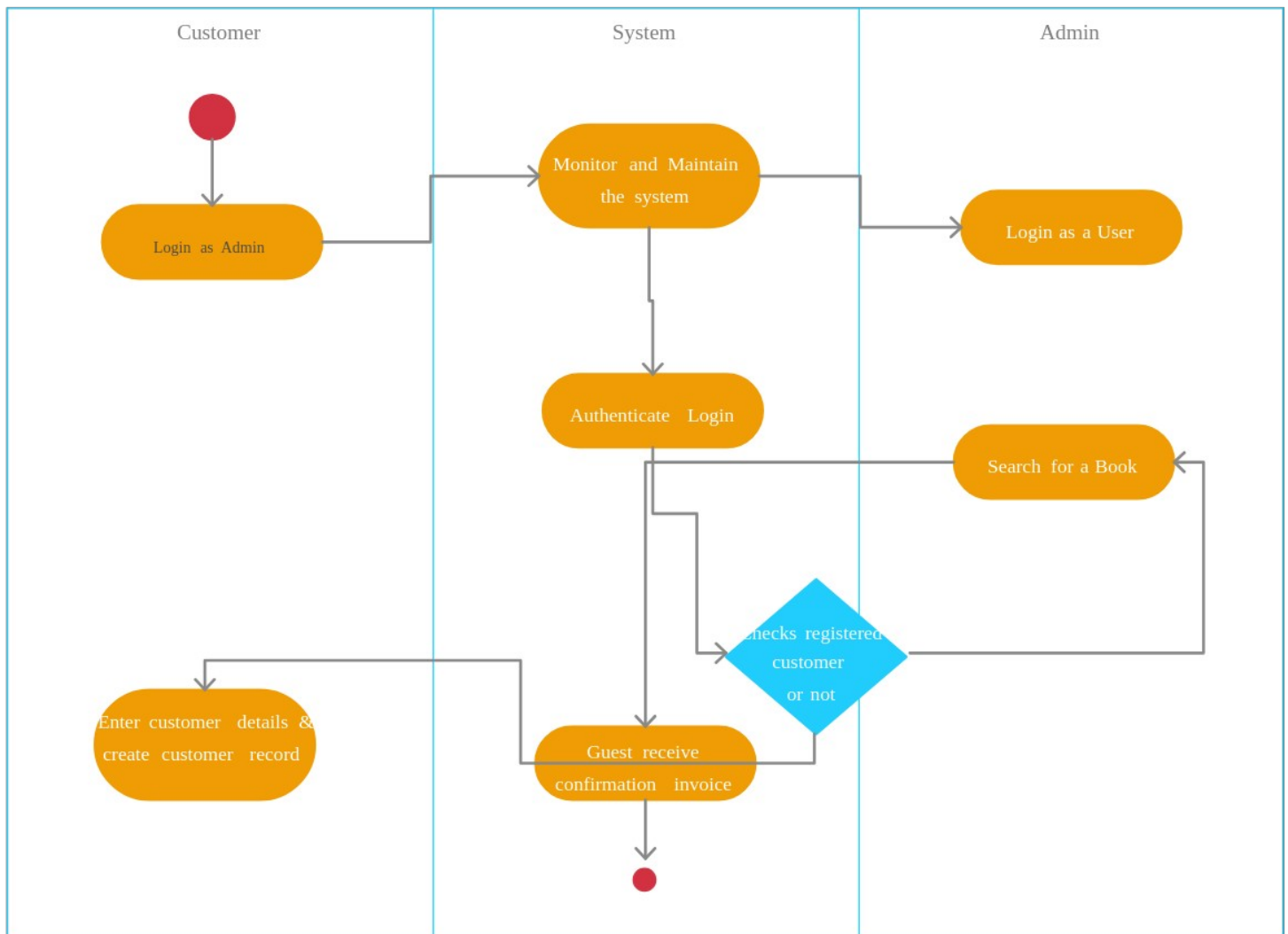
- An activity diagram is a special case of a state diagram in which all (or at least most) of the states are action states and in which all (or at least most) of the transitions are triggered by completion of the actions in the source states. Below are the activity diagrams for the actions performed by the end-user and the response of the system.

5.3 ACTIVITY DIAGRAM:

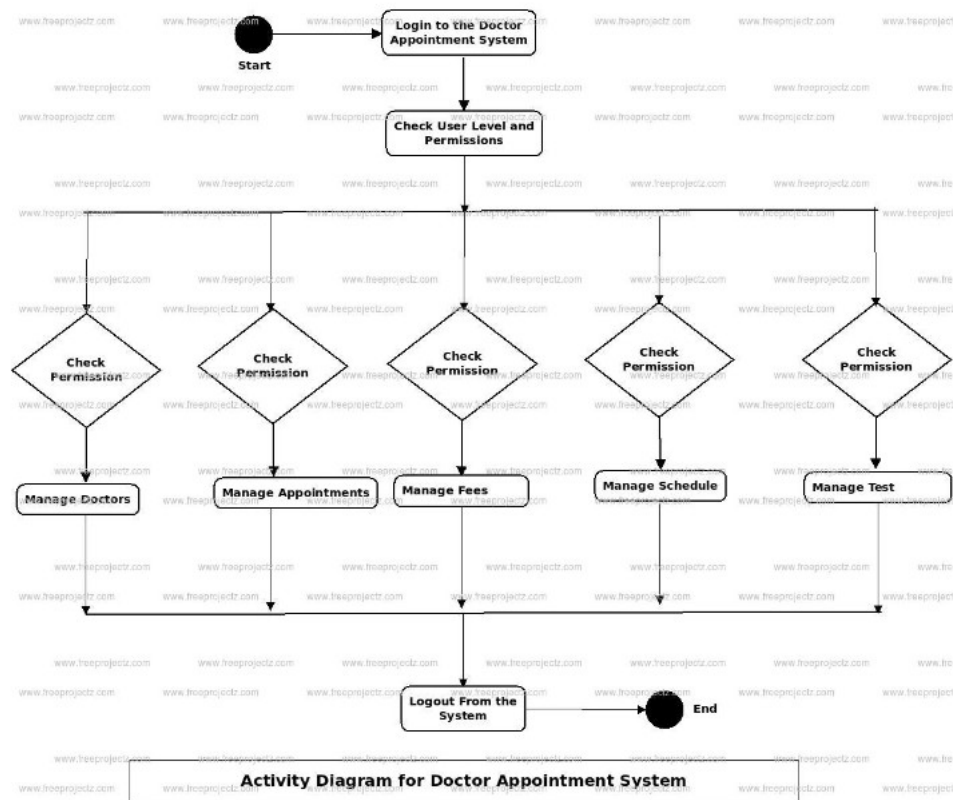
5.3.1 Activity Diagram:



5.3.2 Activity Diagram for customer:



5.3.3 Activity diagram for client:



CHAPTER 6 IMPLEMENTATION STANDERD

a).Implementation Environment:

Challenges identified for successful design and implementation of this project are dominated by: complexity, reliability/availability, transparent data access while respecting security, a high experiment data rate and data volume, scientific exploitation from multiple sites. All the systems have a GUI interface to interact with the user.

i. Multi user:

- This Software is a multi-user Application. This must be uploading on the server and start through internet on which you want to work. Multiple instances of the Application can be run at a time.

ii. GUI based:

- This Software consists of GUI that is user friendly but the background process consists of much center of attraction. That is written in such a way that manipulation of multiple images can be easily done.

b). Security Features:

- Any infrastructure/application development for the web should adhere to the Security Policy detailed by the organization. It is assumed that Organization has a detailed Security policy for protecting its resources and information. What follows is a “common” approach for protecting the resources and information when a web application is published in the web.

Security Layers:

Application Security

Database Security

Network - Data Transfer Security

External Security: Firewall and Antivirus

1) Application Security:

- Single sign-on provides users with a single point of entry to all authorized applications with a single authentication or login process. Application Server Security includes.
 - **Authentication:** System supports strong user Authentication. Server Password-based authentication will support Password-based schemes, to be secure, must facilitate change of passwords regularly, sufficient complexity and cannot be easily guessed.
 - Only Authenticated users can login in the System by entering their own Login Id and Password.
 - We are storing Passwords in Encrypted form so only authenticated user can use that.
- **Authorization:** User Authorization in the system is implemented as access control. Access control deals with the concept of who has access to what information and what type of operations can be accessed. System provides a strong set of access control security mechanisms through privileges.
- **Data Confidentiality:** Encryption is the mechanism that is used to provide data confidentiality. The required data can be sent in encrypted form through SSL (Secure Socket Layer). SSL can be enabled for accessing the applications through Internet.

2) Database Security:

- **Server-Enforced, Row Level Access Control** - The system will be the using Oracle Database. This database will be deployed on the database server and database access checks will be defined at the system level. No direct access is granted to Oracle Database, all access to this database is via applications and hence governed by Access controls.

3) Network - Data Transfer Security

- **Encryption of All Data in Transmission** - encrypts data traveling over the network everywhere.
 - a. Browser to application server
 - b. Application server to database

c. Server to server

- The connection from browser to application server is protected through HTTP's Secure Socket layer with data encryption and encapsulation. In addition to strong user authentication, SSL also provides network data confidentiality and data integrity for multiple types of connections.

4) External Security

- **Anti-Virus Kits:** All the Application servers, database servers and the Web-servers should have the latest anti-virus kits to detect new viruses. These anti-virus kits will be updated with the latest versions frequently.

CHAPTER 7: TESTING

7.1 TESTING PLAN:

7.1.1 Planning steps:

- 1) Functionality Testing
- 2) Usability testing
- 3) Interface testing
- 4) Performance testing
- 5) Security testing

1) Functionality testing:

Test for – all the links in web pages, database connection, forms used in the web pages for submitting or getting information from user, Cookie testing.

● Check all the links:

- Test the outgoing links from all the pages from specific domain under test. Test all internal links. Test links jumping on the same pages.
- Test links used to send the email to admin or other users from web pages. Test to check if there are any orphan pages. Lastly in link checking, check for broken links in all above-mentioned links.

● Test forms in all pages:

- Forms are the integral part of any web site. Forms are used to get information from users and to keep interaction with them. So what should be checked on these forms? First check all the validations on each field. Check for the default values of fields. Wrong inputs to the fields in the forms. Options to create forms if any, form delete, view or modify the forms.
- Let's take example of the search engine project currently I am working on, In this project we have advertiser and affiliate signup steps. Each sign up step is different but dependent on other steps. So sign up flow should get executed correctly. There are different field validations like email Ids, User

financial info validations. All these validations should get checked in manual or automated web testing.

- **Cookies testing:**

Cookies are small files stored on user machine. These are basically used to maintain the session mainly login sessions. Test the application by enabling or disabling the cookies in your browser options. Test if the cookies are encrypted before writing to user machine. If you are testing the session cookies (i.e. cookies expire after the sessions ends) check for login sessions and user stats after session end. Check effect on application security by deleting the cookies. (I will soon write separate article on cookie testing).

- **Validate your HTML/CSS:**

If you are optimizing your site for Search engines then HTML/CSS validation is very important. Mainly validate the site for HTML syntax errors. Check if site is crawl able to different search engines.

- **Database testing:**

Data consistency is very important in web application. Check for data integrity and errors while you edit, delete, modify the forms or do any DB related functionality. Check if all the database queries are executing correctly, data is retrieved correctly and also updated correctly. More on database testing could be load on DB, we will address this in web load or performance testing below.

2) **Usability testing:**

- ▶ **Test for navigation:**

- Navigation means how the user surfs the web pages, different controls like buttons, boxes or how user using the links on the pages to surf different pages. Usability testing includes: Web site should be easy to use. Instructions should be provided clearly. Check if the provided instructions are correct means whether they satisfy purpose. Main menu should be provided on each page. It should be consistent.

- ▶ **Content:**

- Content should be logical and easy to understand. Check for spelling errors. Use of dark colours annoys users and should not be used in site theme. You can follow some standards that are used for web page and content building. These are common accepted standards like as I mentioned above about annoying colors, fonts, frames etc.
- Content should be meaningful. All the anchor text links should be working properly. Images

should be placed properly with proper sizes. These are some basic standards that should be followed in web development. Your task is to validate all for UI testing.

► **Other user information for user help:**

- Like search option, sitemap, help files etc. Sitemap should be present with all the links in web sites with proper tree view of navigation. Check for all links on the sitemap. “Search in the site” option will help users to find content pages they are looking for easily and quickly. These are all optional items and if present should be validated.

3) Interface testing:

- The main interfaces are:
 - Web server and application server interface
 - Application server and Database server interface.
- Check if all the interactions between these servers are executed properly. Errors are handled properly. If database or web server returns any error message for any query by application server then application server should catch and display these error messages appropriately to users. Check what happens if user interrupts any transaction in-between? Check what happens if connection to web server is reset in between?

4) Performance testing:

- Web application should sustain to heavy load. Web performance testing should include:
 - Web Load Testing
 - Web Stress Testing
- **Load testing:** Test application performance on different internet connection speed. In “**web load testing**” test if many users are accessing or requesting the same page. Can system sustain in peak load times? Site should handle many simultaneous user requests, large input data from users, Simultaneous connection to DB, heavy load on specific pages etc.
- **Stress testing:** Generally stress means stretching the system beyond its specification limits. Web stress testing is performed to break the site by giving stress and checked how system reacts to stress and how system recovers from crashes. Stress is generally given on input fields, login and sign up areas. In web performance testing web site functionality on different operating systems, different hardware platforms is checked for software, hardware memory leakage errors.

5) Security testing:

- Following are some test cases for web security testing:
- ⇒ Test by pasting internal url directly into browser address bar without login. Internal pages should not open. If you are logged in using username and password and browsing internal pages then try changing url options directly. I.e. If you are checking some publisher site statistics with publisher site ID= 123.
- ⇒ Try some invalid inputs in input fields like login username, password, input text boxes. Check the system reaction on all invalid inputs.
- ⇒ Web directories or files should not be accessible directly unless given download option. Test if SSL is used for security measures. If used proper message should get displayed when user switch from non-secure http:// pages to secure https:// pages and vice versa. All transactions, error messages, security breach attempts should get logged in log files somewhere on web server.

7.2 TESTING STRATEGIES: WHITE BOX TESTING:

- White box testing (WBT) is also called **Structural or Glass box testing**. White box testing involves looking at the structure of the code. When you know the internal structure of a product, tests can be conducted to ensure that the internal operations performed according to the specification. And all internal components have been adequately exercised.

⇒ **Why do we White box testing?**

⇒ **To ensure:**

- ⇒ That all independent paths within a module have been exercised at least once. All logical decisions verified on their true and false values. All loops executed at their boundaries and within their operational bounds internal data structures validity.

⇒ **Need of White box testing?**

⇒ **To discover the following types of bugs:**

- Logical error tend to creep into our work when we design and implement functions, conditions or controls that are out of the program. The design errors due to difference between logical flow of the program and the actual implementation

➤ **Limitation of WBT:**

Not possible for testing each and every path of the loops in program. This means exhaustive testing is impossible for large systems. This does not mean that WBT is not effective. By selecting important logical paths and data structure for testing is practically possible and effective.

➤ **Black box testing:**

Black box testing treats the system as a “**black-box**”, so it doesn’t explicitly use Knowledge of the internal structure or code. Or in other words the Test engineer need not know the internal working of the “Black box” or application.

- **Main focus in black box testing is on functionality of the system as a whole.** The term ‘**behavioral testing**’ is also used for black box testing and white box testing is also sometimes called ‘**structural testing**’. Behavioral test design is slightly different from black-box test design because the use of internal knowledge isn’t strictly forbidden, but it’s still discouraged. Each testing method has its own advantages and disadvantages. There are some bugs that cannot be found using only black box or only white box. Majority of the application are tested by black box testing method. We need to cover majority of test cases so that most of the bugs will get discovered by black box testing.
- Black box testing occurs throughout the software development and Testing life cycle i.e. in Unit, Integration, System, Acceptance and regression testing stages.

Advantage of Black box testing:

- Tester can be non-technical.
- Used to verify contradictions in actual system and the specifications.
- Test cases can be designed as soon as the functional specifications are complete.

Disadvantage of Black box testing:

- The test inputs needs to be from large sample space.
- It is difficult to identify all possible inputs in limited testing time. So writing test cases is slow and difficult. Chances of having unidentified paths during this testing.

7.3 Test cases:

* Test case for Inconsistent database:

TEST CASE ID: TC001	NAME: Database Inconsistency
PURPOSE Checking out the	consistency of database in oracle server
	through SQL Developer
INPUT	Inserting new data through form
EXPECTED OUTPUT	Reflection/addition of data in database.

STEPS:	
1. Open client payment form	
2. Fill data and click submit it to add to the database	
This should be reflected in database tables.	

*Test case for foreian key constrain:

TEST CASE ID: TC002	NAME: Foreign Key Violation
PURPOSE	To assure that the child record and parent record does not gets
	deleted.
INPUT	Client Details
EXPECTED OUTPUT	Prevention to delete the client details
STEPS:	
1. Select client details form manage client	
2. Click on delete to check violation.	

*Test case for transportation details addition:

TEST CASE ID: TC003	NAME: Check for Database File
----------------------------	--------------------------------------

PURPOSE		To allow addition of Appointment details of call log only if the support is provided onsite.
INPUT		Selection of Onsite option in Support type option
EXPECTED OUTPUT	Allow addition of Appointment details.	
STEPS:		
1. Select Onsite option in support type in call log form		
2. Add Appointment details and click submit.		

CHAPTER 8: LIMITATION AND FUTURE ENHANCEMENT

8.1 LIMITATIONS:

* Following are the limitations of Doctors Appointment System application:

- Some Times Very high data Interaction so server is down.
- Very High security needed to This System.
- It is a step by step procedures i.e., all steps are comes after one another.
- So, every steps is need to be first completed then only we can move the other steps, we need to wait.

8.2 FUTURE ENHANCEMENT :

*Following can be enhancements to Doctors Appointment System application:

- It is simplest and maintained and managed easily, any changes occurs in the future then the changes can be made easily in this system without having ambiguity.
- More Faster searching and Appointment for doctors.
- Not a single website is ever consider as complete forever firstly because here is always something new requirement also are growing day by day. We want to improved our home page, as it is the main thing which attracts all users.

CHAPTER 9: BIBLIOGRAPHY

9.1 Conclusion:

A well-structured doctors' appointment system is crucial for enhancing patient care and optimizing healthcare delivery. By streamlining scheduling processes, reducing wait times, and improving communication, such systems increase patient satisfaction and operational efficiency. Key benefits include better resource management, timely access to care, and improved health outcomes. Ongoing evaluation and adaptation of the system are essential to address challenges and incorporate new technologies, ensuring it meets the evolving needs of patients and healthcare providers.

9.2 Bibliography:

*Web resources

- www.w3schools.com
- www.oracle.com/in/
- www.mysql.com
- www.asp.net

□ BOOKES:

- ASP.NET The complete Reference, McGraw Hill Publication
- SQL, PL/SQL by Ivan Bayross , BPB Publications
- Object Oriented Modelling and Design with UML (second edition) by Michael Blaha and James Rumbaugh
- Software Engineering by Roger S. Pressman
- Software Engineering by K.K. Agrawal
- Oracle 10g SQL and PL/SQL material provided by Oracle University
- Oracle DBA WS-I material by Oracle University.